Concept 9 — Pass by Reference (in Python)

Design question: *When you pass an object into a function, can the function change it permanently?*

The Core Confusion

Python is neither pure "pass by value" nor pure "pass by reference" in the C/Java sense. The accurate term is **pass by object reference** — the function receives a reference to the same object, not a copy.

```
def modify(lst):
    lst.append(4)

numbers = [1, 2, 3]
modify(numbers)
print(numbers)    # [1, 2, 3, 4]
```

The list was changed permanently. `lst` inside the function and `numbers` outside both point to the same object.

But This Behaves Differently

```
def reassign(lst):
    lst = [100, 200]    # this creates a NEW local reference

numbers = [1, 2, 3]
reassign(numbers)
print(numbers)    # [1, 2, 3]  ← unchanged!
```

Why the difference? `lst.append(4)` modifies the object itself. `lst = [100, 200]` doesn't modify the object — it makes `lst` point to a brand new object. The original `numbers` reference outside the function never changes.

The Real Rule

> ***Mutating an object*** *(via methods like `.append()`, `.update()`, or `obj.attr = x`) is visible outside the function — because there's only one object, with multiple references pointing to it.*
> ***Reassigning a variable*** *(`x = something_new`) only changes what that one local variable points to — it does not affect the caller's reference.*

Mutable vs Immutable Objects

This whole topic only matters for **mutable** types — `list`, `dict`, `set`, and custom objects. For **immutable** types — `int`, `str`, `tuple`, `float` — you can never "mutate in place" at all, so reassignment is the only thing that ever happens:

```
def increment(n):
    n += 1

x = 5
increment(x)
print(x)    # 5 — unchanged. Integers are immutable; n += 1 creates a new int.
```

Connecting Back to Concept 2

This is the exact same reference model you already know — `dog` and `another` pointing to the same `Dog` object. Function arguments work identically: the function parameter is just another reference to the same object the caller has.

One-Liner

> *Python passes references to objects, not copies. Mutating the object through that reference is visible to the caller. Reassigning the local variable to a new object is not.*

Concept 10 — Class Diagrams (UML Basics)

Design question: *How do you communicate a class design to someone without showing code?*

Why This Matters

Before writing code, engineers often sketch the design. A **class diagram** is a simple visual notation (from UML — Unified Modeling Language) for showing classes, their attributes, methods, and relationships to each other.

Anatomy of a Class Box

```
┌──────────────────────┐
│        Dog           │   ← Class name
├──────────────────────┤
│ - name: str          │   ← Attributes (with visibility)
│ - age: int           │
├──────────────────────┤
│ + bark(): None       │   ← Methods (with visibility)
│ + eat(): None        │
└──────────────────────┘
```

Visibility symbols:

```
+   public      (accessible from anywhere)
-   private     (only within the class — like Python's __name)
#   protected   (within class and subclasses — like Python's _name)
```

Representing Relationships

Inheritance ("is-a") — hollow triangle arrow, pointing to the parent:

```
   Animal
     △
     |
    Dog
```

Aggregation ("has-a", independent lifecycle) — hollow diamond at the owner:

```
Company ◇──────── Employee
```

Composition ("has-a", owned lifecycle) — filled diamond at the owner:

```
Car ◆──────── Engine
```

Association (general "uses-a" relationship) — plain line:

```
Driver ──────── Car
```

A Worked Example

This single picture tells you: `Dog IS-A Animal` (and overrides `speak()`), and `Company HAS-A` collection of `Employee` objects with an independent lifecycle.

Why Bother With This?

- Communicates design **before** writing code — cheaper to fix a diagram than refactor code
- Forces you to think about relationships explicitly (is this inheritance or aggregation?) before implementation
- Standard enough that any engineer, in any language, can read it

One-Liner

> *A class diagram is a language-agnostic sketch of your design — classes as boxes, inheritance as hollow-triangle arrows, aggregation/composition as diamonds. It's how you think through structure before you touch code.*

Concept 11 — Abstraction

Design question: *How much of an object's internal complexity should the user of that object actually see?*

Abstraction vs Encapsulation — The Constant Confusion

These two are taught together so often that people assume they're the same thing. They are not.

	Encapsulation	Abstraction
Question it answers	*Who* is allowed to change the data?	*How much* of the implementation should the user see?
Mechanism	Access control (`_` , `__` , methods as gatekeepers)	Hiding implementation details behind a simple interface
Focus	Protecting state	Reducing complexity for the user

Encapsulation is about control. **Abstraction** is about simplicity.

A Concrete Example

```
class CoffeeMachine:
    def make_coffee(self):
        self._heat_water()
        self._grind_beans()
        self._brew()
        self._pour()

    def _heat_water(self):
        print("Heating water to 92°C")

    def _grind_beans(self):
        print("Grinding beans")

    def _brew(self):
        print("Brewing under pressure")

    def _pour(self):
        print("Pouring into cup")
```

The user just calls:

```
machine = CoffeeMachine()
machine.make_coffee()
```

They don't need to know about water temperature, grind size, or brew pressure. **That's abstraction** — a simple interface (`make_coffee()`) hiding a complex implementation (four internal steps).

The `_` prefixes on the internal methods are **encapsulation** — signaling these aren't meant to be called directly from outside.

Notice: the same example demonstrates both pillars simultaneously, which is exactly why they get confused. Encapsulation made the internal methods "private-by-convention." Abstraction is the *design decision* to expose only one method (`make_coffee`) as the public interface.

Abstract Base Classes — Python's Formal Tool

Python provides the `abc` module to formally enforce abstraction — defining a contract that subclasses *must* fulfill, without specifying *how*.

```python
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14159 * self.radius ** 2
```

```python
s = Shape()       # TypeError — can't instantiate an abstract class
c = Circle(5)
c.area()           # works — Circle fulfilled the contract
```

`Shape` defines *what* every shape must be able to do (`area()`) without saying *how*. Each subclass decides its own implementation. If `Circle` forgot to implement `area()`, Python would refuse to let you even create a `Circle` object.

Why Abstraction Matters

Without it, every user of your class needs to understand its entire internal machinery to use it safely. With it, they only need to know the public interface — the *what*, not the *how*. This is the same idea behind every library you've ever imported: you call `requests.get(url)` without knowing how TCP sockets work underneath.

One-Liner

> *Abstraction hides how something works behind a simple interface for what it does. Encapsulation controls who can touch the internals. They're often built together, but they answer different questions.*

How These Connect to the Rest

```
Pass by Reference   → How does data move between functions? (mechanics, not a "pillar")
Class Diagrams       → How do you communicate design before writing code?
Abstraction          → How much complexity should the user see? (a true 4th pillar)
```

With this, all four pillars are now covered: **Encapsulation** (Concept 4), **Inheritance** (Concept 8), **Polymorphism** (Concept 8), and **Abstraction** (here).